MedievAIl Battle P2P

Infrastructure réseau pair-à-pair pour IA distribuées
daemon en C, jeu en Python, synchronisation par UDP

Rapport technique

Projet collectif STI3A (projet réseaux) — INSA Centre-Val de Loire — octobre 2025 à avril 2026
Rapport rédigé en octobre 2026 à partir du dépôt, par Mohamed Aziz BAOUEB (@aziz3r), mainteneur du miroir

Code source : https://github.com/aziz3r/medievail-battle-p2p

Résumé

MedievAIl Battle P2P prolonge le jeu de bataille médiéval développé pour le projet Python de la formation : deux joueurs, chacun avec son armée et son IA, s'affrontent sans serveur central. Chaque machine exécute deux processus : le jeu (Python, pygame) et un daemon réseau écrit en C qui relaie les messages en UDP vers le pair. Ce rapport décrit l'architecture, le protocole de lobby et de synchronisation, l'historique du développement, puis une expérience réalisée en octobre 2026 : une partie complète entre deux instances locales, enregistrée côte à côte. L'expérience montre un fonctionnement de bout en bout (lobby, placement, combat) mais aussi une divergence des deux vues en fin de bataille, et révèle une limite propre à macOS : le daemon d'origine perd les paquets de plus de 9 Ko. Le rapport confronte enfin la documentation au code et propose des pistes d'amélioration. Le projet est une œuvre collective ; l'historique Git d'origine est conservé.

Table des matières

1 Contexte et objectifs 1

2 Architecture 2

3 Protocole et déroulement d'une partie 3

4 Historique du développement 4

5 Expérience : une partie complète entre deux instances locales 5

6 Analyse : documentation et code 6

7 Reproductibilité 8

8 Pistes d'amélioration 8

1 Contexte et objectifs

Ce dépôt est un miroir, maintenu par @aziz3r, du projet réseaux de la formation STI3A de l'INSA Centre-Val de Loire (dépôt de référence : YOUSSEF4315/STI3A-Projet_reseaux_1 ; voir `AUTHORS.md` pour les onze contributeurs et leurs droits). Il repose sur la base de jeu du projet Python / POO, décrite dans le dépôt medievail-battle : les deux historiques sont communs jusqu'au

27 janvier 2026. Le code réseau est l'apport de ce dépôt ; il a été écrit entre le 8 et le 29 avril 2026 (21 commits) et le dépôt compte 132 commits d'équipe au total.

L'objectif, tel que l'énonce le README, est de fournir une *infrastructure réseau décentralisée* pour un jeu de bataille stratégique, avec une séparation stricte des responsabilités : la logique de jeu, les IA et l'interface restent dans le processus Python ; la communication entre pairs est confiée à un processus C. L'intérêt pédagogique est de manipuler sockets UDP, communication inter-processus (IPC), cohérence d'un état partagé sans serveur et défauts typiques des réseaux non fiables.

Les chiffres cités sans référence ont été mesurés en octobre 2026 sur un Mac (Python 3.9.6, pygame 2.6.1, compilateur de macOS) ; ils sont indicatifs. Le code compte environ 6 500 lignes de Python (1 972 pour le modèle, 2 134 pour la vue, 1 616 pour le présentateur et 762 à la racine) et 177 lignes de C.

2 Architecture

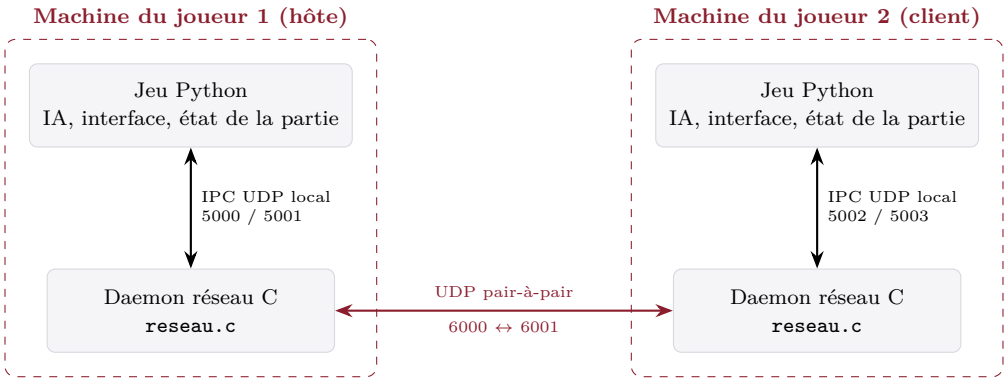


Figure 1 – Topologie : chaque joueur exécute deux processus. Le jeu ne parle qu'à son daemon local ; seuls les daemons C communiquent à travers le réseau. Les numéros sont ceux du README (test sur une seule machine).

2.1 Le daemon réseau (`reseau.c`)

Le daemon est un relais UDP à une seule boucle `select()`. Il se lance avec cinq paramètres (`reseau <port_p2p_local> <ip_distante> <port_p2p_distant> <ipc_entrant> <ipc_sortant>`) et surveille deux sockets : un socket local (`127.0.0.1`, port IPC entrant) et un socket réseau (`0.0.0.0`, port P2P). Le comportement est le suivant :

— *jeu* → *réseau* : tout datagramme reçu sur le port IPC entrant est transmis tel quel au pair, et le daemon renvoie un accusé local (`{"type": "ack"}`) ;

— *réseau* → *jeu* : tout datagramme reçu sur le port P2P est renvoyé au jeu local, sur le port IPC sortant ;

— l'adresse du pair est mise à jour avec l'adresse source du dernier paquet reçu (IP et port), ce qui permet de découvrir le pair dynamiquement.

Le tampon de lecture est de 64 Ko. Le code est portable (Winsock sous Windows, sockets POSIX ailleurs).

2.2 Le pont Python (`network_ipc.py`)

La classe `IPCClient` ouvre deux sockets UDP : un pour envoyer au daemon (port IPC entrant) et un, non bloquant, pour recevoir sur le port IPC sortant. Les messages sont des dictionnaires sérialisés en JSON ; `receive()` lit au plus un datagramme par appel, sans jamais bloquer le jeu. Les tampons d'envoi et de réception sont portés à 65 535 octets. Un routeur de secours en Python (`p2p_node_mock.py`), de même interface, remplace le daemon C quand aucun compilateur n'est disponible ; c'est lui qui sert pour la « version 1 » du README.

2.3 Ports

Joueur	Port P2P (daemon)	Port du pair	IPC entrant	IPC sortant
Joueur 1 (hôte)	6000	6001	5000	5001
Joueur 2 (client)	6001	6000	5002	5003

Table 1 – Configuration pour tester les deux joueurs sur une même machine (les ports IPC diffèrent pour éviter tout conflit).

3 Protocole et déroulement d'une partie

3.1 Lobby

Depuis le menu, « Multijoueur (P2P) » ouvre un lobby : chaque joueur choisit son IA et l'une des quatre zones de déploiement (quadrants de la carte 120×120), puis crée la partie (hôte) ou la rejoint (client). Le jeu envoie alors toutes les 0,5 seconde un message de choix et attend celui de l'adversaire, pendant 10 secondes au plus :

```
{"type": "setup_choice", "ia": "General STRATEGUS", "zone": 1}
```

Chaque processus crée ensuite *localement* les deux armées : 150 unités par camp (50 piquiers, 50 chevaliers, 50 arbalétriers, sur deux colonnes de 25 lignes chacun), placées dans le quadrant choisi. L'équipe locale est `A` pour l'hôte et `B` pour le client, et chaque unité reçoit un identifiant stable `A_1`, `A_2`, … Seule l'IA locale est instanciée : l'adversaire est piloté à distance.

Zone	x	y
1 (haut-gauche)	10–50	10–50
2 (haut-droite)	70–110	10–50
3 (bas-gauche)	10–50	70–110
4 (bas-droite)	70–110	70–110

Table 2 – Quadrants de déploiement (`QUADRANT_BOUNDS` dans `model/army_compositions.py`).

3.2 Placement et démarrage

La partie s'ouvre en phase de *placement* : le jeu ne simule rien (pas de temps `dt=0`) mais continue de recevoir l'état du pair ; un clic gauche ajoute un chevalier à l'armée locale. La touche `ENTRÉE` démarre le combat *pour le joueur local seulement* : aucun message n'est échangé pour démarrer les deux simulations ensemble.

3.3 Synchronisation de l'état

Pendant la partie, chaque instance appelle `Game.step` à chaque image. Au début du pas, elle lit au plus un datagramme du daemon et l'applique ; à la fin du pas, une fois sur cinq, elle diffuse son état compact (exemple de contenu) :

```
{"t": "as", "u": {"A_1": {"tp": "Pikeman", "x": 31.0, "y": 17.0, "h": 55.0},
                  "B_1": {"h": 55.0}, ...}}
```

Pour chaque unité de l'équipe locale (le « propriétaire »), le message contient le type, la position et les points de vie ; pour les unités de l'adversaire, uniquement les points de vie, afin de propager les dégats infligés. À la réception, les positions ne sont appliquées qu'aux unités de l'autre équipe (l'IA locale reste maîtresse des siennes), les unités inconnues sont créées, et les points de vie reçus sont écrits sur toutes les unités.

Taille des paquets. Avec 150 unités par camp, un message complet pèse 12 502 octets au départ, soit 42 octets par unité. À 30 images par seconde, un paquet toutes les 5 images représente 6 paquets par seconde, environ 75 Ko/s dans chaque sens (estimation). Comme la taille croît linéairement avec l'effectif et qu'un datagramme UDP ne peut pas dépasser 65 507 octets, ce schéma est borné à environ 1 500 unités en tout.

4 Historique du développement

L'historique Git permet de suivre la construction de la partie réseau :

Date	Auteur (Git)	Contenu
8 avril 2026	zaydbouazza	Champs réseau de `Guerrier` : identifiant, propriétaire, sérialisation.
9 avril	zaydbouazza	Couche réseau UDP + IPC entre C et Python.
10 avril	Ali ; aziz3r	Intégration de la synchronisation dans `Game` ; lobby multijoueur (IA et zone), corrections de plantages de synchro, UID stables, scripts de test et preuve de concept.
28 avril	Ali, YOUSSEF4315, zaydbouazza	README ; routeur de secours en Python ; simulation de perte de paquets ajoutée puis retirée ; correction de doublons d'unités ; « finalisation V1 » (tampons maximaux, grandes armées).
29 avril	Ali, YOUSSEF4315	Armées de 520 puis de 150 unités par équipe ; bug visuel « mort-vivant » pour la présentation ; README V1/V2 et schémas.

Table 3 – Chronologie du code réseau, d'après les messages de commits (les noms sont ceux qui figurent dans l'historique).

Le README distingue deux versions. La **version 1** est une couche réseau « best-effort » : les actions sont diffusées sans garantie de réception ni d'ordre, ce qui produit des unités dupliquées, des « fantômes » et des désynchronisations (routeur Python, utilisé pour montrer les défauts d'un réseau sans protocole de cohérence). La **version 2** annonce un daemon C avec une notion de *propriété réseau* (une seule entité propriétaire à un instant donné, négociation du transfert avec les pairs), des mécanismes anti-zombie et une victoire synchronisée. La section 6 compare cette description au code présent dans le dépôt.

5 Expérience : une partie complète entre deux instances locales

5.1 Protocole d'expérience

Nous avons exécuté le véritable jeu et le véritable daemon sur une même machine : deux daemons (ports du tableau 1), deux instances du jeu pilotées par un script (souris et clavier simulés, affichage sans fenêtre, temps virtuel à 30 images/s), enregistrées côte à côte. Le joueur 1 (hôte) joue General STRATEGUS en zone 1, le joueur 2 (client) joue Major DAFT en zone 4. Le script et les médias sont dans `demo/` ; il se relance avec `python demo/capture_demo.py`.



Figure 2 – Lobby des deux joueurs après le choix de l'IA et de la zone (hôte à gauche, client à droite).

5.2 Observations

Le trafic passe bien par les daemons C. Sur la partie enregistrée, le daemon de l'hôte a transmis 662 paquets au pair et celui du client 384, ce qui est cohérent avec un paquet toutes les 5 images. Le lobby, le placement des deux armées et le début du combat se déroulent comme décrit : les deux écrans montrent les mêmes armées et chaque joueur voit bouger les unités de l'adversaire (figure 3).

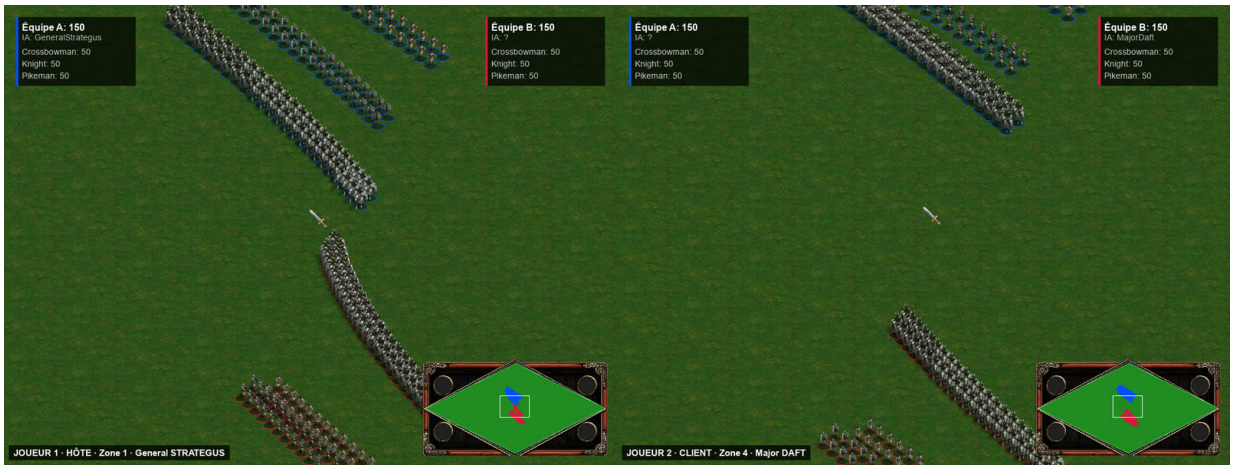


Figure 3 – Les deux armées avancent l'une vers l'autre (même instant du script ; hôte à gauche, client à droite).

Les deux vues finissent par diverger. À la fin de l'enregistrement, le client affiche « Victoire équipe B » (équipe A : 0 unité, équipe B : 112), alors que l'hôte affiche encore équipe A : 1 et équipe

B : 108 et n'a pas détecté de fin de partie. Des unités teintées en vert — le drapeau `is_zombie`, voir la section 6 — apparaissent pendant la mêlée. L'hôte a d'ailleurs joué plus longtemps que le client (117 s contre 77 s enregistrées).

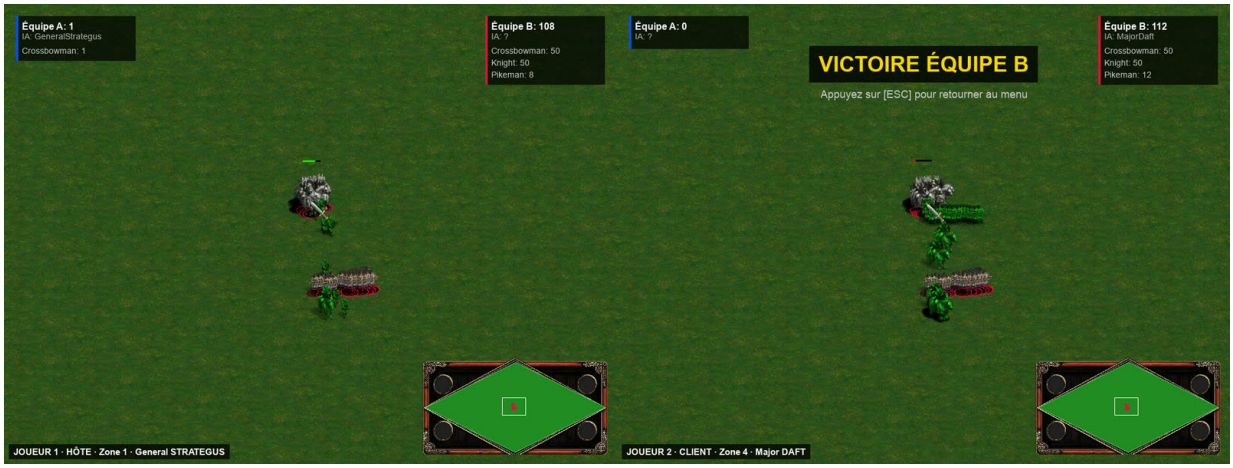


Figure 4 – État final affiché par chaque joueur. Les deux captures sont prises à la fin du script de chaque joueur, donc pas au même instant.

Ces observations proviennent d'une seule exécution. Le script de capture rend les images à des vitesses qui diffèrent d'un joueur à l'autre (rendu sans fenêtre) alors que la synchronisation dépend du rythme des images : cela peut avoir amplifié la divergence, sans que nous puissions la quantifier.

5.3 Un obstacle propre à macOS : la taille maximale d'un datagramme

Sous macOS, la taille maximale d'un datagramme UDP est limitée par défaut à 9 216 octets (`sysctl net.inet.udp.maxdgram`), et `reseau.c` ne règle pas la taille des tampons de ses sockets (`SO_SNDBUF`). Nous avons envoyé au daemon des datagrammes de tailles croissantes depuis `network_ipc.py` et vérifié leur arrivée au pair :

Taille du message	Reçu par le jeu du pair (daemon d'origine)
1 000 octets	oui
8 000 octets	oui
12 000 octets	**non** (perdu)
30 000 octets	**non** (perdu)

Table 4 – Test du daemon d'origine compilé sur macOS (une expérience par taille).

Or le premier message de synchronisation fait 12 502 octets : avec le daemon d'origine, la synchronisation du combat ne peut pas passer sur ce système. Pour la démonstration, le script compile donc une *copie temporaire* de `reseau.c` dans laquelle les tampons des deux sockets sont portés à 1 Mo ; le fichier du dépôt n'est pas modifié. Sous Linux et Windows, la limite par défaut est plus large et le problème ne se pose pas dans les mêmes termes (non testé).

6 Analyse : documentation et code

6.1 La propriété réseau annoncée n'est pas dans le code

Dans les fichiers du dépôt :

— `reseau.c` est un relais : il ne contient ni table de propriété, ni consensus, ni négociation de transfert ;
— le champ `proprietaire_reseau` est défini et sérialisé par `Guerrier.to_dict` / `update_from_dict`, mais ces deux méthodes ne sont jamais appelées ;
— la synchronisation réelle passe par `get_sync_state` / `apply_sync_state`, qui écrit sans arbitrage les points de vie reçus sur n'importe quelle unité. Un commentaire annonce « on prend le minimum pour ne jamais annuler un dégat subi », mais le code qui suit écrit la valeur reçue, et un second commentaire parle de « création du bug de rollback ».

Mécanisme du zombie. Chaque pair diffuse les points de vie de *toutes* les unités qu'il connaît, y compris celles de l'adversaire. Si le pair a une valeur périmée, elle écrase la valeur à jour de l'unité propriétaire : l'unité retrouve des points de vie perdus (rollback), ou *ressuscite* si elle était morte. Le code l'assume : lorsqu'une unité morte localement reçoit des points de vie positifs, elle est marquée `is_zombie` et dessinée en vert. Le commit du 29 avril intitulé « Visual Zombie bug (Mort-Vivant) implementation for presentation » suggère que ce comportement a été ajouté volontairement, pour illustrer le problème lors de la présentation. Le dépôt est donc cohérent avec ce que nous observons ; en revanche, les mécanismes de la version 2 (propriété, anti-zombie, victoire synchronisée) décrits dans le README ne sont pas implémentés dans les fichiers présents.

6.2 Autres limites repérées

— **UDP sans fiabilité applicative.** Pas de numéro de séquence, d'horodatage ni de détection de doublons : chaque paquet contient un état complet, ce qui tolère la perte mais pas le désordre ni les valeurs périmées.
— **Démarrage et fin non synchronisés.** `ENTRÉE` démarre la simulation du seul joueur local, et chaque pair calcule la victoire sur son propre état ; aucun message de début ou de fin n'existe.
— **Une seule lecture par pas.** Le jeu ne lit qu'un datagramme par image ; ce n'est pas bloquant au débit actuel, mais le débit de réception n'est pas contrôlé.
— **Taille bornée.** Voir la section 3.3 : le message croît avec l'effectif et dépasse la limite par défaut de macOS à environ 220 unités.
— **Sécurité.** Aucune authentification ni chiffrement ; le daemon écoute sur toutes les interfaces et adopte comme pair l'expéditeur du dernier paquet reçu ; plusieurs erreurs de réseau sont ignorées (blocs `except` qui se terminent par `pass`). À ne pas exposer en dehors d'un réseau de confiance.
— **Scripts de test.** `verify_sync.py` lance deux daemons avec les ports IPC par défaut, identiques : le second quitte avec l'erreur `bind(): Address already in use` (vérifié). Il appelle aussi `./reseau` alors que le README compile `reseau.exe`, et génère `test_joiner.py` à la volée.
— **Fichiers parasites.** Un exécutable compilé (`network_poc/p2p_node`, binaire macOS arm64 de 34 Ko) et une copie temporaire encodée en UTF-16 (`temp_remote_network_ipc.py`) sont versionnés.
— **Documentation de compilation.** Le README donne la commande Windows (`-lws2_32`) ; sous macOS, `gcc reseau.c -o reseau -lpthread` suffit (la limite des datagrammes mentionnée plus haut s'applique).

7 Reproductibilité

Pour rejouer l'expérience du README sur une machine (quatre terminaux), avec la version 2 :

```
gcc reseau.c -o reseau -lpthread          # Linux / macOS
./reseau 6000 127.0.0.1 6001 5000 5001    # terminal 1 : daemon hote
python launch.py                          # terminal 2 : menu 6, zone 1, CREER
./reseau 6001 127.0.0.1 6000 5002 5003    # terminal 3 : daemon client
python launch.py                          # terminal 4 : menu 6, zone 4, REJOINDRE
```

Sous macOS, les messages de synchronisation de 150 unités par camp dépassent la taille maximale d'un datagramme (section 5.3) ; la démonstration automatisée (`python demo/capture_demo.py`) contourne ce point avec une copie temporaire du daemon.

8 Pistes d'amélioration

1. Implémenter l'arbitrage annoncé : propriétaire unique de chaque unité, dégats transmis comme événements (ou valeur minimale des points de vie) au lieu d'états écrits aveuglément.
2. Ajouter numéros de séquence et horodatage aux messages pour ignorer les paquets périmés ou dupliqués.
3. Ajouter un message de départ (« prêt », puis début commun) et un message de fin de partie pour que les deux joueurs concluent de la même manière.
4. Réduire la taille des messages (différences d'état, entiers courts) ou prévoir la fragmentation, et régler `SO_SNDBUF` / `SO_RCVBUF` dans `reseau.c` pour macOS.
5. Envisager un modèle *lockstep* : échanger seulement les ordres des IA, avec une graine d'aléa commune, pour rendre la simulation déterministe et les messages très petits.
6. Écrire un test automatisé sur l'interface locale (deux `IPCClient`, deux relais) et corriger `verify_sync.py` (ports IPC distincts) ; retirer les fichiers binaires et temporaires du dépôt.